Studio Nx

S1-11c Sampler MIDI 연주 — 함수 설명 합본 (JS 301~304)

2026-10-21

목차

1. ai-notes/javascript/S1-11c - Sampler MIDI 연주/301_audio_sampler.md
2. ai-notes/javascript/S1-11c - Sampler MIDI 연주/302_audio_sampler_wiring.md
3. ai-notes/javascript/S1-11c - Sampler MIDI 연주/303_sequencer_sampler.md
4. ai-notes/javascript/S1-11c - Sampler MIDI 연주/304_sequencer_devices_sampler.md

JavaScript 301 — Sampler 음원 (`audio/sampler.js`)

항목	내용
파일	`studionx/frontend/static/js/audio/sampler.js`
작업	S1-11c Sampler MIDI 연주: MIDI 노트 하나를 샘플로 울리는 그래프와 계산 함수 (FR-241)
작성	Claude (AI), 2026-10-21, 사용자 검토
이전, 다음	이전 300 내보내기 창, 다음 302 Sampler 연결

1. 하는 일

Sampler 악기에 붙은 샘플을 MIDI 노트의 음높이에 맞춰 울립니다. Synth의 `playSynthNote`(291)와 같은 위치에서 같은 모양으로 쓰입니다. 입력은 노트(음높이, 세기, 시작 시각, 길이)와 디코딩된 샘플이고, 출력은 소리를 앞당겨 끊을 때 쓰는 `{ stop }`입니다.

모드가 둘입니다.

모드	소리 나는 방식	무시하는 값
`classic`	건반을 눌러 음높이를 바꿔 치는 방식입니다. 노트 길이만큼 울리고 Release로 끝납니다. Loop를 켜면 샘플을 되풀이합니다.	없음
`oneshot`	드럼처럼 한 번 치면 샘플 전체가 울립니다.	노트 길이, Loop, Sustain, Release

```
playSamplerNote(ctx, dest, p, buffer, pitch, velocity, when, duration)
  buffer 없음 —▶ 아무것도 만들지 않고 빈 { stop } 을 돌려줍니다
  AudioBufferSourceNode(샘플, playbackRate) —▶ GainNode(앰프 엔벨로프) —▶ dest(악기 층)
    classic : Attack → Decay → Sustain, 노트 끝에서 Release, Loop 가 켜져 있으면 샘플을 되풀이
    oneshot : Attack, Decay 만 적용하고 샘플이 끝날 때까지 울림
```

2. 언제 쓰나

쓰는 곳	하는 일
`InstrumentRack.playNote` (302)	트랙의 악기 층이 Sampler이면 이 함수를 불러 노트를 울립니다.
`session.js`, `offline.js` (302)	`samplerSampleIds`로 미리 디코딩할 샘플 번호를 모읍니다.
시험 `sampler_unit.mjs`, `render_sampler.mjs`	계산과 노드 연결, 실제 소리를 확인합니다.

3. 코드

3.1 순수 함수

```
export function samplerMode(value) {
  return value === 'oneshot' ? 'oneshot' : 'classic';
}
```

```
export function samplerRate(pitch, root, tune) {
  return Math.pow(2, (Number(pitch) - Number(root) + (Number(tune) || 0)) / 12);
}
```

```
export function samplerSampleIds(getInstruments, trackIds) {
  const out = [];
  (trackIds || []).forEach(function (trackId) {
    (getInstruments(trackId) || []).forEach(function (inst) {
      if (inst.plugin_code !== 'sampler' || inst.is_enabled === false) return;
      const params = inst.params || {};
      const id = params.sample_id;
      if (id == null || id === '') return;
      const n = Number(id);
      if (isFinite(n) && out.indexOf(n) < 0) out.push(n);
    });
  });
  return out;
}
```

3.2 노트 재생

```
export function playSamplerNote(ctx, dest, p, buffer, pitch, velocity, when, duration) {
  if (!buffer) return { stop: function () {} };
  const mode = samplerMode(p.mode);
  const rate = samplerRate(pitch, p.root_note != null ? p.root_note : 60, p.tune);
  const peak = Math.max(0, Number(p.gain) || 0) * Math.max(0, Math.min(127, velocity)) / 127;
  const a = Math.max(0, Number(p.attack) || 0);
  const d = Math.max(0, Number(p.decay) || 0);

  const src = ctx.createBufferSource();
  src.buffer = buffer;
  src.playbackRate.value = rate;

  const amp = ctx.createGain();
  src.connect(amp);
  amp.connect(dest);

  let end;
  amp.gain.setValueAtTime(0, when);
  if (mode === 'oneshot') {
    // 노트 길이를 무시하고 샘플 전체를 울림. Attack 은 앞을 부드럽게 열고, Decay 가 있으면 그 시간 동안 0
까지 줄임 (드럼의 꼬리 조절)
    if (a > 0) amp.gain.linearRampToValueAtTime(peak, when + a);
    else amp.gain.setValueAtTime(peak, when);
    if (d > 0) amp.gain.linearRampToValueAtTime(0, when + a + d);
    end = when + buffer.duration / rate + 0.02;
    src.start(when);
    src.stop(end);
  } else {
    const s = p.sustain != null ? Number(p.sustain) : 1;
    const r = Math.max(0.005, Number(p.release) || 0);
    const dur = Math.max(0.01, duration);
    if (a > 0) amp.gain.linearRampToValueAtTime(peak, when + a);
    else amp.gain.setValueAtTime(peak, when);
    if (d > 0 && dur > a) amp.gain.linearRampToValueAtTime(peak * s, when + Math.min(dur, a + d));
```

```
    amp.gain.setValueAtTime(envelopeLevel(dur, a, d, s, peak), when + dur);
    amp.gain.linearRampToValueAtTime(0, when + dur + r);
    end = when + dur + r + 0.02;
    if (p.loop === true) {
      src.loop = true;
      src.loopStart = 0;
      src.loopEnd = buffer.duration;
    }
    src.start(when);
    src.stop(end);
  }

  return {
    // 정지 · 루프 되감기 때 소리를 끊음 (뚝 끊기는 소리를 줄이려고 아주 짧은 릴리스를 줌)
    stop: function (at) {
      const t = Math.max(at, ctx.currentTime);
      try { amp.gain.cancelScheduledValues(t); amp.gain.setTargetAtTime(0, t, 0.01); } catch (e) { /* 무
시 */ }
      try { src.stop(t + 0.08); } catch (e) { /* 이미 끝남 */ }
    },
  };
}
```

4. 재생 속도 계산

눌린 건반이 Root(샘플이 원래 내는 음)에서 몇 반음 떨어졌는지에 Tune을 더해서 2의 거듭제곱으로 바꿉니다. 한 옥타브가 12반음이고 주파수가 2배가 되므로, 반음 하나는 2의 12제곱근(약 1.0595)배입니다.

눌린 건반	Root	Tune	재생 속도	들리는 소리
60	60	0	1배	샘플 그대로
72	60	0	2배	한 옥타브 위, 길이는 절반
48	60	0	0.5배	한 옥타브 아래, 길이는 두 배
60	60	+12	2배	Tune으로 한 옥타브 위
60	48	0	2배	Root를 낮추면 같은 건반이 더 높게 울림

`playbackRate`는 음높이와 속도를 함께 바꾸므로, 높은 음일수록 샘플이 빨리 끝납니다. 오래된 하드웨어 샘플러와 같은 동작이고, 드럼이나 짧은 소리에는 이 방식이 가장 자연스럽습니다.

5. 최신 문법으로 쓰면

```
// samplerSampleIds 를 최신 문법으로 쓴 모양
export const samplerSampleIds = (getInstruments, trackIds = []) => [...new Set(
  trackIds
    .flatMap((id) => getInstruments(id) ?? [])
    .filter((inst) => inst.plugin_code === 'sampler' && inst.is_enabled !== false)
    .map((inst) => inst.params?.sample_id)
    .filter((id) => id != null && id !== '')
    .map(Number)
    .filter(Number.isFinite),
)];
```

```
// 음량 계산
const peak = Math.max(0, p.gain ?? 0) * Math.min(127, Math.max(0, velocity)) / 127;
```

전통 문법 (실제 코드)	최신 문법
`out.indexOf(n) < 0`로 중복을 걸러 `push`	`new Set`과 전개 `[...set]`
`forEach` 안에서 `forEach`	`flatMap`
`(getInstruments(trackId) || [])`	`getInstruments(trackId) ?? []`
`params.sample_id` 앞에 `inst.params || {}`	`inst.params?.sample_id`

6. 왜 이렇게

선택	이유
샘플이 없으면 빈 `{ stop }`을 돌려줍니다	샘플을 아직 고르지 않았거나 불러오지 못한 악기도 노트 재생이 멈추지 않아야 합니다. 호출하는 쪽이 `stop`을 항상 부를 수 있게 같은 모양을 지킵니다.
`oneshot`은 샘플 길이를 재생 속도로 나눠 끝 시각을 정합니다	속도가 2배이면 샘플이 절반 시간에 끝납니다. 길이를 그대로 쓰면 `stop` 시각이 너무 늦어서 쓸데없이 노드가 남습니다.
`oneshot`에서 Attack과 Decay는 남겨 둡니다	Attack은 앞의 클릭 소리를 줄이고, Decay가 있으면 그 시간에 걸쳐 0까지 줄여 드럼의 꼬리를 짧게 만들 수 있습니다. Sustain과 Release는 노트 길이에 의존하는 값이라 무시합니다.
`classic`의 엔벨로프는 `envelopeLevel`(291)을 다시 씁니다	노트가 Attack이나 Decay 도중에 끝나도 Release가 그 자리의 값에서 시작해야 소리가 튀지 않습니다. Synth와 같은 계산이므로 같은 함수를 씁니다.
같은 샘플을 연속으로 쳐도 앞의 소리를 끊지 않습니다	노트마다 `AudioBufferSourceNode`를 새로 만들기 때문에 자연스럽게 겹쳐 울립니다. 하이햇을 빠르게 칠 때처럼 앞의 소리의 꼬리가 남는 것이 실제 장비와 같은 소리입니다. 결정 문서(`Default_Instruments_Effects.md`)에서 정한 동작입니다.
계산을 순수 함수로 따로 둡니다	`samplerRate`와 `samplerSampleIds`는 소리를 쓰지 않으므로 Node에서 바로 시험할 수 있습니다.
재생과 내보내기가 같은 함수를 씁니다	`AudioBufferSourceNode`는 `OfflineAudioContext`에서도 똑같이 동작하므로, 내보낸 WAV에도 Sampler 소리가 자동으로 들어갑니다.

7. 시험

- `docs/handoff/s1-11c_tests/sampler_unit.mjs` (39건)는 Node에서 순수 함수와 노드 연결을 확인합니다.
  - 모드 이름: 알 수 없는 값은 `classic`입니다.
  - 재생 속도: 같은 음 1배, 한 옥타브 위 2배, 아래 0.5배, 7반음 위 약 1.4983, Tune ±12, Root 변경, 소수 Tune, 숫자가 아닌 Tune입니다.
  - 샘플 번호 모음: 중복 제거, Sampler가 아닌 악기 무시, `null` 건너뜀, 꺼진 층 건너뜀, 문자열 번호를 숫자로, 빈 입력입니다.
  - 노드 연결: 기록하는 가짜 컨텍스트로, `classic`의 정지 시각(노트 길이 + Release + 0.02초), Loop 구간, 음량(gain × velocity/127), `oneshot`의 정지 시각(샘플 길이 ÷ 속도), Loop와 Release 무시, Attack과 Decay 램프를 확인합니다.
- `docs/handoff/s1-11c_tests/render_sampler.mjs` (28건)는 실제 `OfflineAudioContext`로 소리를 그려서 숫자로 확인합니다.
  - A: 음높이가 440Hz, 880Hz, 220Hz로 나오고 Root와 Tune이 반영됩니다.
  - B: Release, Attack이 소리 크기에 반영됩니다.

- C: Loop가 꺼지면 샘플 길이(0.5초)에서 끝나고, 켜지면 노트가 끝날 때까지 이어지며 440Hz가 유지됩니다.
- D: `oneshot`은 노트가 아주 짧아도 샘플 끝까지 울리고, Loop, Sustain, Release를 무시하며, Decay로 꼬리가 줄고, 음높이는 따릅니다.
- E: 같은 샘플을 연속으로 치면 앞의 소리가 이어서 울려 합쳐집니다.
- F: gain 0.5는 피크가 절반, velocity 64는 64/127입니다.
- G: 샘플이 없거나 불러오지 못해도 오류 없이 조용합니다.
- H: Sampler와 Synth가 한 트랙에 있으면 둘 다 울리고, 같은 샘플은 한 번만 불러옵니다.

JavaScript 302 — Sampler 연결 (`instruments.js`, `session.js`, `offline.js`)

항목	내용
파일	`studionx/frontend/static/js/audio/instruments.js`, `studionx/frontend/static/js/audio/session.js`, `studionx/frontend/static/js/audio/offline.js`
작업	S1-11c Sampler MIDI 연주: 악기 층에서 Sampler를 울리고, 재생과 내보내기가 Sampler의 샘플을 미리 불러옵니다 (FR-241)
작성	Claude (AI), 2026-10-21, 사용자 검토
이전, 다음	이전 301 Sampler 음원, 다음 303 샘플 고르기

1. 하는 일

301의 `playSamplerNote`가 소리를 내려면 두 가지가 더 필요합니다. 하나는 트랙의 악기 층이 Sampler를 알아보고 그 함수를 부르는 것이고, 다른 하나는 노트가 울리기 전에 샘플이 디코딩되어 있는 것입니다. 이 문서는 그 연결을 설명합니다.

```
노트 → NotePlayer → InstrumentRack.playNote(트랙, 음높이, ...)
                      각 악기 층마다
                        plugin_code === 'sampler' ─▶ getBuffer(params.sample_id) 로 샘플을 찾아 playSamplerNote
                        그 밖 (synth_sub)         ─▶ playSynthNote

재생 시작: PlaybackSession.start        ┐ 클립의 샘플 + samplerSampleIds 로 구한
내보내기:  renderOffline                ┘ Sampler 의 샘플을 함께 디코딩
```

2. 언제 쓰나

쓰는 곳	하는 일
`PlaybackSession._ensure` (290)	`InstrumentRack`을 만들 때 세 번째 뒤에 `getBuffer`(재생 세션이 담아 둔 샘플)를 넘깁니다.
`PlaybackSession.start` (290)	오디오 클립의 샘플과 Sampler의 샘플을 함께 불러온 뒤에 재생을 시작합니다.
`renderOffline` (299)	같은 방식으로 Sampler의 샘플을 디코딩하고, `InstrumentRack`에 `getBuffer`를 넘깁니다.

3. 코드

3.1 악기 층 (`instruments.js`)

```
export class InstrumentRack {
  /**
   * @param {BaseAudioContext} context
   * @param {function(number|string):AudioNode} trackInput  트랙 번호 → 입력 노드 (MixerGraph.trackInput)
   * @param {function(number|string):Array<object>} getInstruments  트랙 번호 → 악기 층 목록
   *   층: { id, plugin_code, schema, params, volume_db, pan, is_enabled }
   * @param {function(number|string):(AudioBuffer|null)} [getBuffer]  샘플 번호 → 디코딩된 샘플 (Sampler가 씀)
```

```
   */
  constructor(context, trackInput, getInstruments, getBuffer) {
    this.context = context;
    this.trackInput = trackInput;
    this.getInstruments = getInstruments;
    this.getBuffer = getBuffer ? getBuffer : function () { return null; };
    this.layers = {};   // 층 번호 → { gain, pan }
  }

  /** 층의 출력 노드를 (없으면) 만들고 볼륨 · 팬을 맞춤 */
  _layer(trackId, inst) {
    const key = String(trackId) + ':' + String(inst.id);
    const ctx = this.context;
    let layer = this.layers[key];
    if (!layer) {
      const gain = createStereoGain(ctx);
      const pan = typeof ctx.createStereoPanner === 'function' ? ctx.createStereoPanner() : null;
      if (pan) { gain.connect(pan); pan.connect(this.trackInput(trackId)); }
      else gain.connect(this.trackInput(trackId));
      layer = { gain: gain, pan: pan };
      this.layers[key] = layer;
    }
    layer.gain.gain.value = layerGain(inst.volume_db);
    if (layer.pan) layer.pan.pan.value = Math.max(-1, Math.min(1, Number(inst.pan) || 0));
    return layer;
  }

  /**
   * 노트 하나를 트랙의 모든 악기 층에 울림
   * @returns {Array<{stop:function(number):void}>} 울린 소리 (층마다 하나)
   */
  playNote(trackId, pitch, velocity, when, durationSec) {
    const out = [];
    const self = this;
    (this.getInstruments(trackId) || []).forEach(function (inst) {
      if (inst.is_enabled === false) return;
      if (PLAYABLE_CODES.indexOf(inst.plugin_code) < 0) return;
      const layer = self._layer(trackId, inst);
      const params = resolveParams(inst.schema, inst.params);
      if (inst.plugin_code === 'sampler') {
        const buffer = params.sample_id != null ? self.getBuffer(params.sample_id) : null;
        out.push(playSamplerNote(self.context, layer.gain, params, buffer, pitch, velocity, when, durati
onSec));
      } else {
        out.push(playSynthNote(self.context, layer.gain, params, pitch, velocity, when, durationSec));
      }
    });
    return out;
  }
}
```

3.2 재생 시작 때 샘플 불러오기 (`session.js`)

```
async start(fromTick) {
  this._ensure();
  const self = this;
```

```
  const deps = this.deps;
  this.clips = deps.getClips();
  this.noteEvents = expandNotes(deps.getMidiClips ? deps.getMidiClips() : []);
  const ids = this.clips.filter(function (c) { return c.sample_id != null; }).map(function (c) { return
c.sample_id; });
  // Sampler 악기가 쓰는 샘플도 함께 불러옴 (S1-11c)
  const trackIds = deps.getTracks().map(function (t) { return t.track_id; });
  samplerSampleIds(deps.getInstruments ? deps.getInstruments : function () { return []; }, trackIds)
    .forEach(function (id) { if (ids.indexOf(id) < 0) ids.push(id); });
  await Promise.all(ids.map(function (id) { return self._loadSample(id); }));

  this.mixer.applyTracks(deps.getTracks());
  this.effects.sync(deps.getTracks());
  this.engine.setTempo(deps.getBpm(), deps.getTempoChanges ? deps.getTempoChanges() : []);
  if (deps.getMetronome) this.engine.setMetronome(deps.getMetronome());
  const loop = deps.getLoop ? deps.getLoop() : null;
  this.engine.setLoop(loop ? loop.start : 0, loop ? loop.end : 0, !!loop);

  this.player.stopAll();
  this.notePlayer.stopAll();
  this.lastTick = null;
  this.engine.start({ fromTick: fromTick });
  this.playing = true;
  // 시작 위치에 걸쳐 있는 클립은 중간부터 바로 울림
  const t = this.engine.transport;
  clipsCovering(this.clips, t.originTick).forEach(function (c) {
    self.player.schedule(c, t.originTick, t.originTime, self.engine.transport.baseBpm, self.engine.trans
port.changes);
  });
}
```

3.3 내보내기의 샘플 디코딩 (`offline.js`)

아래 함수 안에서 `samplerSampleIds`로 Sampler의 샘플 번호를 `ids`에 더하는 부분과, `InstrumentRack`을 만들 때 `getBuffer`를 넘기는 한 줄이 이번에 추가되었습니다.

```
export async function renderOffline(o) {
  const changes = o.tempoChanges || [];
  const progress = o.onProgress || function () {};
  const cancelled = o.isCancelled || function () { return false; };
  const Ctx = o.OfflineContext || (typeof OfflineAudioContext !== 'undefined' ? OfflineAudioContext : nu
ll);
  if (!Ctx) throw new Error('이 브라우저는 OfflineAudioContext 를 지원하지 않습니다');

  const rangeSec = ticksToSecondsWithChanges(o.endTick, o.bpm, changes) - ticksToSecondsWithChanges(o.st
artTick, o.bpm, changes);
  const totalSec = rangeSec + Math.max(0, o.tailSec || 0);
  const frames = Math.max(1, Math.ceil(totalSec * o.sampleRate));
  const ctx = new Ctx(2, frames, o.sampleRate);

  const audioClips = clipsInRange(o.audioClips, o.startTick, o.endTick);
  const midiClips = clipsInRange(o.midiClips, o.startTick, o.endTick);

  // 1) 샘플 디코딩 (오프라인 컨텍스트의 샘플레이트로 다시 맞춰짐). 전체 진행률의 앞 10%
  const buffers = {};
```

```
  const ids = [];
  audioClips.forEach(function (c) { if (c.sample_id != null && ids.indexOf(String(c.sample_id)) < 0) id
s.push(String(c.sample_id)); });
  // Sampler 악기가 쓰는 샘플도 함께 디코딩 (S1-11c)
  samplerSampleIds(o.getInstruments, o.tracks.map(function (t) { return t.track_id; })).forEach(function
(id) {
    if (ids.indexOf(String(id)) < 0) ids.push(String(id));
  });
  for (let i = 0; i < ids.length; i++) {
    if (cancelled()) throw new Error('cancelled');
    try {
      const bytes = await o.loadSampleBytes(Number(ids[i]));
      buffers[ids[i]] = await ctx.decodeAudioData(bytes);
    } catch (err) {
      if (typeof console !== 'undefined') console.warn('[export] 샘플 ' + ids[i] + ' 을(를) 불러오지 못해
건너뜁니다', err);
    }
    progress(0.1 * (i + 1) / ids.length);
  }

  // 2) 재생과 같은 그래프
  const mixer = new MixerGraph(ctx);
  const effects = new EffectRack(ctx, mixer, o.getEffects, function () { return o.bpm; });
  const rack = new InstrumentRack(ctx, function (id) { return mixer.trackInput(id); }, o.getInstruments,
function (sid) { return buffers[String(sid)] || null; });
  const player = new ClipPlayer(ctx, function (id) { return mixer.trackInput(id); }, function (sid) { re
turn buffers[String(sid)] || null; });
  const notePlayer = new NotePlayer(ctx, function (id, pitch, vel, when, dur) { return rack.playNote(id,
pitch, vel, when, dur); });
  effects.sync(o.tracks);
  mixer.applyTracks(o.tracks, true);

  // 3) 창마다 소리를 예약
  const noteEvents = expandNotes(midiClips).filter(function (ev) { return ev.abs_start >= o.startTick;
});
  const bounds = windowBoundaries(o.startTick, o.endTick, o.bpm, changes, WINDOW_SECONDS);
  const scheduleWindow = function (k) {
    const a = bounds[k];
    const b = bounds[k + 1];
    const nowTime = ticksToSecondsWithChanges(a, o.bpm, changes) - ticksToSecondsWithChanges(o.startTic
k, o.bpm, changes);
    if (k === 0) {
      // 시작 위치에 걸쳐 있는 클립은 중간부터 울림
      clipsCovering(audioClips, a).forEach(function (c) { player.schedule(c, a, nowTime, o.bpm, change
s); });
    }
    clipsStartingIn(audioClips, a, b).forEach(function (c) { player.schedule(c, a, nowTime, o.bpm, chang
es); });
    notesStartingIn(noteEvents, a, b).forEach(function (ev) { notePlayer.schedule(ev, a, nowTime, o.bpm,
changes); });
  };
  scheduleWindow(0);

  // 4) 창 경계마다 멈춰서 다음 창을 예약. 경계 시각은 렌더링 샘플 단위로 맞춰짐
  const t0 = ticksToSecondsWithChanges(o.startTick, o.bpm, changes);
  const stops = [];
  for (let k = 1; k < bounds.length - 1; k++) {
```

```
    const sec = ticksToSecondsWithChanges(bounds[k], o.bpm, changes) - t0;
    if (sec > 0 && sec < totalSec) stops.push({ k: k, sec: sec });
  }
  let failed = null;
  stops.forEach(function (s) {
    ctx.suspend(s.sec).then(function () {
      if (failed) { ctx.resume(); return; }
      if (cancelled()) { failed = new Error('cancelled'); ctx.resume(); return; }   // 남은 구간은 소리
없이 빠르게 끝내고 결과를 버림
      scheduleWindow(s.k);
      progress(0.1 + 0.9 * (s.sec / totalSec));
      ctx.resume();
    });
  });

  const rendered = await ctx.startRendering();
  if (failed) throw failed;
  progress(1);
  const channels = [];
  for (let c = 0; c < rendered.numberOfChannels; c++) channels.push(rendered.getChannelData(c));
  return { channels: channels, sampleRate: rendered.sampleRate, frames: rendered.length };
}
```

4. 바뀐 점

위치	바뀐 내용
`PLAYABLE_CODES`	`['synth_sub']`에서 `['synth_sub', 'sampler']`로 늘었습니다. 이 목록에 없는 악기는 노트를 받아도 아무 소리도 내지 않습니다.
`InstrumentRack` 생성자	네 번째 인자 `getBuffer(sampleId)`를 받습니다. 없으면 항상 `null`을 돌려주는 함수로 대신해서, 이전 호출(인자 세 개)도 그대로 동작합니다.
`InstrumentRack.playNote`	악기 코드가 `sampler`이면 `getBuffer`로 샘플을 찾아 `playSamplerNote`를, 아니면 `playSynthNote`를 부릅니다.
`PlaybackSession.start`	클립의 샘플 번호에 `samplerSampleIds`로 구한 번호를 더하고, 중복 없이 모두 불러온 뒤에 재생합니다.
`renderOffline`	같은 번호 모음을 디코딩하고, `buffers` 객체를 `getBuffer`로 넘깁니다.

5. 최신 문법으로 쓰면

```
// 재생 시작 때 불러올 샘플 번호 모으기를 최신 문법으로 쓴 모양
const clipIds = this.clips.filter((c) => c.sample_id != null).map((c) => c.sample_id);
const trackIds = deps.getTracks().map((t) => t.track_id);
const ids = [...new Set([...clipIds, ...samplerSampleIds(deps.getInstruments ?? (() => []), trackId
s)])];
await Promise.all(ids.map((id) => this._loadSample(id)));
```

전통 문법 (실제 코드)	최신 문법
`forEach` 안에서 `indexOf`로 중복을 걸러 `push`	`new Set`과 전개
`deps.getInstruments ? deps.getInstruments : function () { return []; }`	`deps.getInstruments ?? (() => [])`

전통 문법 (실제 코드)	최신 문법
`function (t) { return t.track_id; }`	화살표 함수 `(t) => t.track_id`

6. 왜 이렇게

선택	이유
샘플 저장소를 `InstrumentRack`이 갖지 않고 `getBuffer` 함수로 받습니다	재생 세션은 `buffers`를, 오프라인 렌더링은 자기 `buffers`를 씁니다. 저장소를 함수로 주입하면 `InstrumentRack`은 어느 쪽인지 몰라도 되고, 컨텍스트를 인자로 받는 기존 설계(S1-11a)와 같은 방식입니다.
노트가 울리기 전에 샘플을 모두 불러옵니다	노트를 울리는 시점에는 비동기로 기다릴 수 없습니다. 재생 시작 때 모든 샘플을 불러오고 시작하는 기존 클립 처리(290)와 같은 원칙입니다.
꺼진 층과 `sample_id`가 없는 층은 불러오지 않습니다	쓰지 않을 샘플을 내려받고 디코딩하는 것은 낭비입니다. 샘플이 없는 Sampler는 소리만 나지 않을 뿐 오류가 아닙니다.
샘플을 못 불러와도 재생과 내보내기는 계속합니다	한 파일 때문에 전체가 멈추지 않게 하는 기존 방침(290, 299)을 그대로 따릅니다. `getBuffer`가 `null`을 돌려주면 301이 조용히 건너뜁니다.
같은 샘플은 한 번만 불러옵니다	클립과 Sampler가 같은 샘플을 쓰거나, 여러 층이 같은 샘플을 써도 `ids`에서 중복을 걸러 한 번만 디코딩합니다. 시험 H에서 확인합니다.
인자를 맨 뒤에 선택으로 추가했습니다	기존 시험 9개(115건)가 `InstrumentRack`을 인자 세 개로 만들고 있습니다. 호출하는 쪽을 고치지 않아도 되게 해서 회귀가 없음을 보장했습니다.

7. 시험

- 위 `sampler_unit.mjs`의 `PLAYABLE_CODES`와 `InstrumentRack` 부분이 목록과 `getBuffer` 호출을 확인합니다.
- `render_sampler.mjs`의 G와 H가 샘플을 불러오지 못한 경우와, 같은 샘플을 쓰는 두 층이 샘플을 한 번만 불러오는 것을 확인합니다.
- `docs/handoff/s1-11c_tests/ui_sampler.mjs` (39건)가 실제 화면에서 재생과 내보내기를 확인합니다.
  - 재생: 샘플이 없으면 조용하고, 있으면 노트 시작 직후 소리가 나며, `oneshot`은 노트(0.5초)가 끝난 뒤에도 샘플이 끝까지 울리고, `classic`은 노트가 끝나고 Release가 지나면 조용합니다.
  - 내보내기: 같은 두 경우가 `renderOffline` 결과에서도 똑같이 나옵니다.
- 기존 시험(S1-11a 9개, S1-11b 3개)은 모두 그대로 통과했습니다. 시험 중 발견한 문제는 없었습니다.

JavaScript 303 — 샘플 고르기 (`pages/sequencer-sampler.js`)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-sampler.js`
작업	S1-11c Sampler MIDI 연주: 상세 창의 Sample 줄, 샘플 고르기 창, 카드 부제 (FR-241)
작성	Claude (AI), 2026-10-21, 사용자 검토
이전, 다음	이전 302 Sampler 연결, 다음 304 장치 화면 연결

1. 하는 일

Sampler 악기에 어떤 샘플을 붙일지 고르는 화면 부품을 모았습니다. 목업(`docs/page_mockups/sampler/`)에서 정한 모양을 그대로 만듭니다.

부품	내용
Sample 줄	고른 샘플의 이름, 길이, 샘플레이트, 채널을 한 줄로 보여 주고 Change... 버튼을 둡니다. 샘플이 없으면 "No sample chosen"과 Choose... 버튼, 경고 문구가 나옵니다.
파형과 Root 표시	샘플의 파형을 그리고 왼쪽 위에 Root 노트 이름(예: Root C4)을 붙입니다.
고르기 창	Change...를 누르면 줄 아래에 열립니다. 라이브러리의 샘플을 이름으로 검색하고, 줄마다 미리 듣기(▶) 버튼이 있습니다.
끌어 놓기	왼쪽 브라우저의 샘플 줄을 Sample 줄 위로 끌어 놓으면 그 샘플로 바뀝니다.
카드 부제	카드 이름 아래에 "샘플 이름, 모드"를 보여 주고, 샘플이 없으면 경고를 보여 줍니다.

화면 부품과 DOM이 없는 계산을 한 파일에 두었고, 계산은 Node에서도 시험합니다.

2. 언제 쓰나

쓰는 곳	하는 일
`sequencer-devices.js`의 `cardHtml` (304)	`cardSubtitle`과 `sampleInfoOf`로 카드 부제를 만듭니다.
`sequencer-devices.js`의 `paramCellHtml`, `bindModalParam` (304)	`type: "sample"` 칸을 `samplePickHtml`로 그리고 `bindSamplePick`으로 동작을 연결합니다.
`sequencer-devices.js`의 `renderModalParams` (304)	`ignoredKeys`로 One-Shot에서 무시되는 칸을 흐리게 합니다.
`sequencer-devices.js`의 `initDevices` (304)	`loadSampleLibrary`로 샘플 목록을 미리 읽어 둡니다.

3. 코드

3.1 순수 함수

```
export function noteName(n) {
  const v = Math.round(Number(n) || 0);
```

```
  return NOTE_NAMES[((v % 12) + 12) % 12] + (Math.floor(v / 12) - 1);
}
```

```
export function ignoredKeys(mode) {
  return mode === 'oneshot' ? ['loop', 'sustain', 'release'] : [];
}
```

```
export function cardSubtitle(info, sampleId, modeLabel) {
  if (sampleId == null || sampleId === '') return { empty: true, text: 'No sample chosen. Notes make no
sound.' };
  const name = info ? info.name : 'Sample #' + sampleId;
  return { empty: false, text: name + ' · ' + modeLabel };
}
```

```
export function sampleInfoText(info) {
  if (!info) return '';
  const parts = [];
  if (info.duration_ms) {
    const sec = Math.round(info.duration_ms / 1000);
    parts.push(Math.floor(sec / 60) + ':' + (sec % 60 < 10 ? '0' : '') + (sec % 60));
  }
  if (info.sample_rate) parts.push((info.sample_rate / 1000) + ' kHz');
  if (info.channels) parts.push(info.channels === 1 ? 'mono' : (info.channels === 2 ? 'stereo' : info.ch
annels + ' ch'));
  return parts.join(' · ');
}
```

3.2 샘플 목록

```
export function loadSampleLibrary(force) {
  if (loading && !force) return loading;
  loading = (async function () {
    const all = [];
    for (let page = 1; page <= MAX_PAGES; page += 1) {
      const res = await fetchSampleList({ page: page, size: PAGE_SIZE });
      const items = res && res.items ? res.items : [];
      items.forEach(function (it) { all.push(it); });
      if (all.length >= (res ? res.total : 0) || !items.length) break;
    }
    library = all;
    return library;
  }()).catch(function (err) {
    loading = null;
    if (window.console) console.warn('[sampler] 샘플 목록을 읽지 못했습니다', err);
    return library;
  });
  return loading;
}
```

```
export function sampleInfoOf(sampleId) {
  if (sampleId == null || sampleId === '') return null;
  const found = library.filter(function (s) { return String(s.sample_id) === String(sampleId); })[0];
  return found || null;
}
```

3.3 Sample 줄

```
export function samplePickHtml(item, value, rootNote) {
  const has = value != null && value !== '';
  const info = has ? sampleInfoOf(value) : null;
  const name = has ? (info ? info.name : 'Sample #' + value) : 'No sample chosen';
  const meta = has ? sampleInfoText(info) : '';
  return '<div class="sq-param sq-param--wide sq-samp' + (has ? '' : ' is-empty') + '" data-param-cell
="' + esc(item.key) + '">'
    + '<span class="sq-samp__label">' + esc(item.name) + '</span>'
    + '<div class="sq-samp__row">'
    + '<span class="sq-samp__icon" aria-hidden="true">♬</span>'
    + '<span class="sq-samp__name">' + esc(name) + '</span>'
    + '<span class="sq-samp__meta sq-num">' + esc(meta) + '</span>'
    + '<button type="button" class="sq-dialog__btn" data-samp="change">' + (has ? 'Change…' : 'Choose…')
+ '</button></div>'
    + (has
      ? '<div class="sq-samp__wave" data-samp="wave"><canvas width="560" height="56" aria-label="Wavefor
m of ' + esc(name) + '"></canvas>'
        + '<span class="sq-samp__root">Root ' + esc(noteName(rootNote)) + '</span></div>'
      : '<p class="sq-samp__warn">⚠ No sample chosen. Notes make no sound.</p>')
    + '<div class="sq-samp__pick" data-samp="pick" hidden>'
    + '<strong>Choose a sample from your library</strong>'
    + '<input type="search" data-samp="search" placeholder="Search samples…" aria-label="Search sample
s">'
    + '<ul data-samp="list"></ul>'
    + '<small>You can also drag a sample from the browser onto the waveform.</small></div>'
    + '</div>';
}
```

```
function renderPickList(cell, query, selectedId) {
  const list = cell.querySelector('[data-samp="list"]');
  if (!list) return;
  const q = String(query || '').trim().toLowerCase();
  const found = library.filter(function (s) { return !q || String(s.name).toLowerCase().indexOf(q) >= 0;
});
  if (!found.length) {
    list.innerHTML = '<li class="sq-samp__none">' + (library.length ? 'No sample matches.' : 'Your libra
ry is empty. Upload a sample in the browser first.') + '</li>';
    return;
  }
  list.innerHTML = found.map(function (s) {
    const tag = s.tags && s.tags.length ? ' · ' + esc(s.tags[0].name) : '';
    return '<li><button type="button" class="sq-samp__item' + (String(s.sample_id) === String(selectedI
d) ? ' is-on' : '') + '" data-sample-id="' + s.sample_id + '">'
      + '<span aria-hidden="true">♬</span><span class="sq-samp__iname">' + esc(s.name) + '</span>'
      + '<span class="sq-samp__imeta sq-num">' + esc(sampleInfoText(s).split(' · ')[0] || '') + tag +
'</span></button>'
      + '<button type="button" class="sq-samp__prev" data-preview="' + s.sample_id + '" aria-label="Prev
iew ' + esc(s.name) + '">▶</button></li>';
  }).join('');
}
```

```
async function paintWave(cell, sampleId) {
  const canvas = cell.querySelector('canvas');
```

```
  if (!canvas || !canvas.getContext) return;
  let peaks = peaksCache[sampleId];
  if (!peaks) {
    try {
      const res = await fetchSampleWaveform(sampleId);
      peaks = res && res.peaks ? res.peaks : [];
      peaksCache[sampleId] = peaks;
    } catch (err) {
      return;   // 파형이 없어도 샘플은 울립니다
    }
  }
  const g = canvas.getContext('2d');
  const w = canvas.width;
  const h = canvas.height;
  g.clearRect(0, 0, w, h);
  g.fillStyle = window.getComputedStyle(canvas).color;
  if (!peaks.length) return;
  for (let x = 0; x < w; x += 1) {
    const p = peaks[Math.min(peaks.length - 1, Math.floor(x * peaks.length / w))];
    const lo = Math.max(-1, Math.min(1, Number(p[0]) || 0));
    const hi = Math.max(-1, Math.min(1, Number(p[1]) || 0));
    const y1 = (1 - hi) / 2 * h;
    const y2 = (1 - lo) / 2 * h;
    g.fillRect(x, y1, 1, Math.max(1, y2 - y1));
  }
}
```

```
async function previewSample(sampleId) {
  if (previewAudio) {
    const same = previewAudio.dataset.sampleId === String(sampleId);
    previewAudio.pause();
    previewAudio = null;
    if (same) return;
  }
  const url = await fetchSampleFileUrl(sampleId);
  previewAudio = new Audio(url);
  previewAudio.dataset.sampleId = String(sampleId);
  previewAudio.addEventListener('ended', function () { previewAudio = null; });
  await previewAudio.play();
}
```

```
export function bindSamplePick(cell, value, onPick) {
  const pick = cell.querySelector('[data-samp="pick"]');
  const search = cell.querySelector('[data-samp="search"]');
  const change = cell.querySelector('[data-samp="change"]');
  if (value != null && value !== '') paintWave(cell, value);

  change.addEventListener('click', function () {
    const open = pick.hidden;
    pick.hidden = !open;
    if (!open) return;
    renderPickList(cell, '', value);
    loadSampleLibrary(true).then(function () { renderPickList(cell, search.value, value); });
    search.value = '';
    search.focus();
  });
```

```
  search.addEventListener('input', function () { renderPickList(cell, search.value, value); });
  pick.addEventListener('click', function (e) {
    const prev = e.target.closest('[data-preview]');
    if (prev) {
      previewSample(prev.getAttribute('data-preview')).catch(function () { /* 재생이 막혀도 고르기는 계속
*/ });
      return;
    }
    const item = e.target.closest('[data-sample-id]');
    if (item) {
      stopSamplePreview();
      onPick(Number(item.getAttribute('data-sample-id')));
    }
  });

  // 브라우저의 샘플 줄을 칸 위에 놓으면 그 샘플로 바꿈
  cell.addEventListener('dragover', function (e) {
    if (e.dataTransfer && Array.prototype.indexOf.call(e.dataTransfer.types, DND_TYPE) >= 0) {
      e.preventDefault();
      cell.classList.add('is-drop');
    }
  });
  cell.addEventListener('dragleave', function () { cell.classList.remove('is-drop'); });
  cell.addEventListener('drop', function (e) {
    cell.classList.remove('is-drop');
    if (!e.dataTransfer) return;
    let data = null;
    try { data = JSON.parse(e.dataTransfer.getData(DND_TYPE)); } catch (err) { data = null; }
    if (!data || data.kind !== 'audio' || !data.sampleId) return;
    e.preventDefault();
    onPick(Number(data.sampleId));
  });
}
```

4. 동작 순서

```
상세 창을 열면 Sample 칸을 samplePickHtml 이 그리고, bindSamplePick 이 동작을 붙입니다
  샘플이 있으면 —▶ paintWave: GET /samples/{id}/waveform 의 peaks 를 캔버스에 세로선으로 그림
  Change… 클릭 —▶ 고르기 창이 열리고 loadSampleLibrary(true) 로 목록을 새로 읽어 그림
  검색어 입력  —▶ renderPickList 가 이름에 검색어가 든 샘플만 다시 그림
  ▶ 클릭       —▶ previewSample: 파일 주소를 받아 재생, 같은 샘플을 다시 누르면 멈춤
  샘플 클릭    —▶ onPick(번호): 호출한 쪽이 초안(draft)을 바꾸고 칸을 다시 그림 (Save 를 눌러야 서버에 저
장)
  브라우저 줄을 칸 위에 놓으면 —▶ 같은 onPick
```

노트 이름은 MIDI 번호 60을 C4로 읽습니다. `noteName(60)`은 `'C4'`, `noteName(61)`은 `'C#4'`, `noteName(48)`은 `'C3'`입니다.

5. 최신 문법으로 쓰면

```
// noteName 을 최신 문법으로 쓴 모양
const NOTE_NAMES = ['C', 'C#', 'D', 'D#', 'E', 'F', 'F#', 'G', 'G#', 'A', 'A#', 'B'];
```

```
export const noteName = (n) => {
  const v = Math.round(Number(n) || 0);
  return `${NOTE_NAMES[((v % 12) + 12) % 12]}${Math.floor(v / 12) - 1}`;
};

// 샘플 정보 찾기와 검색
export const sampleInfoOf = (id) => (id == null || id === '' ? null : library.find((s) => String(s.sampl
e_id) === String(id)) ?? null);
const found = library.filter((s) => !q || s.name.toLowerCase().includes(q));
```

전통 문법 (실제 코드)	최신 문법
`'C' + 4`처럼 `+`로 문자열 잇기	템플릿 리터럴 `` `${a}${b}` ``
`filter(...)[0]`로 첫 항목 찾기	`find`
`indexOf(q) >= 0`	`includes(q)`
`found || null`	`found ?? null`
`res && res.items ? res.items : []`	`res?.items ?? []`

6. 왜 이렇게

선택	이유
새 파일로 분리했습니다	`sequencer-devices.js`가 이미 약 2300줄입니다. 샘플 고르기는 서로 연결된 부품(줄, 고르기 창, 파형, 미리 듣기, 끌어 놓기)이라 하나로 묶으면 읽기 쉽습니다.
샘플 목록을 한 번 읽어 담아 둡니다	카드마다 이름을 보여 주려고 요청을 보내면 낭비입니다. 목록은 한 쪽 50개씩 최대 10쪽까지 읽고, 고르기 창을 열 때만 다시 읽어서 방금 올린 샘플도 보이게 합니다.
목록을 못 읽어도 카드는 "Sample #번호"로 보입니다	이름이 안 보일 뿐 소리와 저장에는 문제가 없으므로, 오류를 띄우지 않고 콘솔에만 경고합니다.
파형은 서버의 `peaks`를 씁니다	브라우저에서 샘플 전체를 다시 디코딩하지 않아도 됩니다. 브라우저의 샘플 줄(S1-07)도 같은 API를 씁니다. 파형을 못 받아도 샘플은 울립니다.
파형 요청은 샘플마다 한 번만 합니다	모드를 바꾸거나 노브를 만져서 칸을 다시 그릴 때마다 요청하지 않도록 `peaksCache`에 담습니다.
샘플을 고르는 것은 초안만 바꿉니다	다른 파라미터와 똑같이 Save를 눌러야 저장되고, Reset과 A/B 비교도 그대로 동작합니다.
끌어 놓기는 브라우저의 기존 형식을 씁니다	왼쪽 목록이 이미 `application/x-studionx-clip`으로 샘플 번호를 싣고 있습니다. 오디오가 아닌 항목(MIDI 패턴)은 무시합니다.
미리 듣기는 하나만 재생합니다	여러 개를 연달아 누르면 소리가 겹치므로, 새로 누르면 이전 것을 멈춥니다. 상세 창을 닫을 때도 멈춥니다(`stopSamplePreview`).

7. 시험

- `docs/handoff/s1-11c_tests/ui_sampler.mjs` (39건)가 헤드리스 Chrome으로 실제 화면을 확인합니다.
  - 카드: 샘플이 없으면 "No sample chosen. Notes make no sound." 경고와 점선 테두리가 나오고, 고른 뒤에는 "샘플 이름, One-Shot"으로 바뀝니다.
  - 상세 창: 탭이 Sample과 Amp Env이고, Sample 줄과 Mode 줄이 있으며, Root 노브가 C4로 보입니다.
  - 고르기 창: 시험 샘플 2개가 목록에 있고, 줄마다 미리 듣기 버튼이 있으며, 검색하면 거릅니다.

    - 고른 뒤: 이름과 정보 줄(0:02, 44.1 kHz, mono), 그려진 파형, Root C4 표시, 경고 사라짐을 확인합니다.
    - 끌어 놓기: 샘플이 없는 상태에서도 놓으면 붙고 안내 문구("Drag a sample here...")가 보이며, 샘플 줄을 놓으면 바뀌고, MIDI 패턴을 놓으면 무시되며, Reset이 기본값으로 되돌립니다.
- 시험 중 발견한 일: 첫 실행에서 샘플 이름 기대값이 `s1-11c-sine440.wav`였는데, 라이브러리는 확장자를 뗀 `s1-11c-sine440`으로 보여 줍니다. 화면이 맞고 시험의 기대값이 틀렸습니다. 또 이 화면은 트랙의 펼침 상태를 기억해 두었다가(`restoreOpenPanels`) 새로 열 때 복원하므로, 시험이 펼침 버튼을 무조건 누르면 오히려 접힙니다. 닫혀 있을 때만 누르도록 고쳤습니다.

JavaScript 304 — 장치 화면의 Sampler 연결 (`pages/sequencer-devices.js`)

항목	내용
파일	`studionx/frontend/static/js/pages/sequencer-devices.js`, `studionx/frontend/static/css/pages/studio.css`
작업	S1-11c Sampler MIDI 연주: 카드 부제와 경고, Mode 버튼 줄, Sample 줄 연결, One-Shot에서 무시되는 칸 표시 (FR-241)
작성	Claude (AI), 2026-10-21, 사용자 검토
이전, 다음	이전 303 샘플 고르기, 다음 없음 (S1-11c의 마지막 문서)

1. 하는 일

악기 카드와 상세 창은 `param_schema`로 자동 생성됩니다(S1-09). 그런데 Sampler에는 노브, 셀렉트, 토글로는 표현할 수 없는 칸이 둘 생겼습니다. 샘플을 고르는 칸과, 목업처럼 이어 붙은 버튼으로 보여 주는 모드 칸입니다. 이 문서는 자동 생성 UI에 그 둘을 더한 부분을 설명합니다.

스키마의 새 항목	그려지는 모양
`type: "sample"` (`sample_id`)	303의 Sample 줄
`type: "select"` + `display: "segment"` (`mode`)	`Classic`, `One-Shot` 버튼 줄과, 고른 옵션의 `hint` 설명
`unit: "note"` (`root_note`)	노브 값을 숫자(60) 대신 노트 이름(C4)으로 표시

스키마에 필드를 더한 것이므로 다른 플러그인이 `display: "segment"`나 `hint`를 쓰면 같은 모양으로 그려집니다.

2. 언제 쓰나

쓰는 곳	하는 일
악기 칸을 그릴 때 `cardHtml`	Sampler 카드에 부제와 경고를 붙입니다.
카드를 눌러 상세 창을 열 때 `renderModalParams`	칸을 그리고 동작을 연결한 뒤, One-Shot에서 무시되는 칸을 흐리게 합니다.
시험 `ui_sampler.mjs`	카드, 상세 창, 저장, 소리를 실제 화면에서 확인합니다.

3. 코드

3.1 카드

```
export function cardHtml(kind, track, d) {
  const p = pluginById(d.plugin_id);
  const name = p ? p.name : 'Plugin #' + d.plugin_id;
  const values = withDefaults(p, d.params);
  const prim = primaryItems(p);
  const bypassed = kind === 'EFFECT' && d.is_bypassed;
  const idAttr = kind === 'EFFECT'
```

```
    ? ' data-track-plugin-id="' + d.track_plugin_id + '"'
    : ' data-track-instrument-id="' + d.track_instrument_id + '"';
  const knobs = prim.map(function (item) {
    return '<span class="sq-knob" role="slider" tabindex="0" data-param-key="' + esc(item.key) + '"'
      + ' aria-label="' + esc(item.name) + '" aria-valuemin="' + numOr(item.min, 0) + '"'
      + ' aria-valuemax="' + numOr(item.max, 1) + '" aria-valuenow="' + numOr(values[item.key], 0) +
'">'
      + '<i style="--a:' + knobAngle(item, values[item.key]) + 'deg"></i>'
      + '<span class="sq-knob__val">' + esc(fmtValue(item, values[item.key])) + '</span>'
      + '<span class="sq-knob__name">' + esc(item.name) + '</span></span>';
  }).join('');
  const mix = kind === 'INSTRUMENT' ? instrumentMixHtml(d) : effectExtraHtml();
  // Sampler 는 이름 아래에 "샘플 이름 · 모드" 를, 샘플이 없으면 경고를 보여 줌 (S1-11c)
  let sub = '';
  let noSample = false;
  if (kind === 'INSTRUMENT' && isSampler(p)) {
    const info = cardSubtitle(sampleInfoOf(values.sample_id), values.sample_id, selectLabel(schemaItem
(p, 'mode'), values.mode));
    noSample = info.empty;
    sub = '<span class="sq-fx__sub' + (info.empty ? ' is-warn' : '') + '">' + (info.empty ? '⚠ ' : '') +
esc(info.text) + '</span>';
  }
  return '<li class="sq-fx sq-fx--dev' + (bypassed ? ' is-bypassed' : '') + (noSample ? ' is-nosample' :
'') + '" draggable="' + String(kind === 'EFFECT') + '"'
    + ' data-device-kind="' + kind + '"' + idAttr + '>'
    + '<span class="sq-fx__art" aria-hidden="true">' + artHtml(p) + '</span>'
    + '<div class="sq-fx__head"><span class="sq-fx__grip" aria-hidden="true">' + (kind === 'EFFECT' ?
'⠿' : '⏻') + '</span>'
    + '<span class="sq-fx__name">' + esc(name) + '</span>'
    + (kind === 'EFFECT'
      ? '<button type="button" class="sq-fx__power" aria-pressed="' + String(!d.is_bypassed) + '" aria-l
abel="Bypass">⏻</button>'
      : '')
    + '<button type="button" class="sq-fx__del" aria-label="Remove ' + (kind === 'EFFECT' ? 'effect' :
'instrument') + '">✕</button></div>'
    + sub
    + '<div class="sq-knobs">' + knobs + '</div>'
    + '<span class="sq-fx__preset">Preset: ' + esc(d.__preset || 'Init') + '</span>'
    + mix + '</li>';
}
```

3.2 상세 창의 칸

```
export function renderModalParams() {
  const box = document.getElementById('device-params');
  if (!box || !modal) return;
  const schema = (modal.plugin.param_schema || []).filter(function (item) {
    return (item.group || 'General') === modal.group;
  });
  box.innerHTML = schema.map(function (item) { return paramCellHtml(item); }).join('')
    || '<p class="sq-cell__hint">This plugin has no parameters.</p>';
  schema.forEach(function (item) {
    const cell = box.querySelector('[data-param-cell="' + item.key + '"]');
    if (cell) bindModalParam(cell, item);
  });
  // One-Shot 모드에서 무시되는 칸은 흐리게 (S1-11c)
```

```
  const ignored = isSampler(modal.plugin) ? ignoredKeys(modal.draft.mode) : [];
  ignored.forEach(function (key) {
    const cell = box.querySelector('[data-param-cell="' + key + '"]');
    if (cell) {
      cell.classList.add('is-ignored');
      cell.title = 'Ignored in One-Shot mode';
    }
  });
}
```

```
function segmentHtml(item, v) {
  const opts = item.options || [];
  const cur = opts.filter(function (o) { return String(o.value) === String(v); })[0];
  const buttons = opts.map(function (o) {
    const on = String(o.value) === String(v);
    return '<button type="button" data-seg="' + esc(o.value) + '" class="' + (on ? 'is-on' : '') + '" ar
ia-pressed="' + String(on) + '">' + esc(o.label) + '</button>';
  }).join('');
  return '<div class="sq-param sq-param--wide sq-segcell" data-param-cell="' + esc(item.key) + '">'
    + '<span class="sq-samp__label">' + esc(item.name) + '</span>'
    + '<div class="ex-seg" role="group" aria-label="' + esc(item.name) + '">' + buttons + '</div>'
    + (cur && cur.hint ? '<p class="sq-segcell__hint">' + esc(cur.hint) + '</p>' : '')
    + '</div>';
}
```

```
function paramCellHtml(item) {
  const v = modal.draft[item.key];
  if (item.type === 'sample') return samplePickHtml(item, v, modal.draft.root_note);
  if (item.type === 'select' && item.display === 'segment') return segmentHtml(item, v);
  let control;
  if (item.type === 'select') {
    const opts = (item.options || []).map(function (o) {
      return '<option value="' + esc(o.value) + '"' + (String(o.value) === String(v) ? ' selected' : '')
+ '>' + esc(o.label) + '</option>';
    }).join('');
    control = '<select data-modal-param="' + esc(item.key) + '" aria-label="' + esc(item.name) + '">' +
opts + '</select>';
  } else if (item.type === 'toggle') {
    control = '<span class="sq-switch" role="switch" tabindex="0" data-modal-param="' + esc(item.key) +
'" aria-label="' + esc(item.name) + '" aria-pressed="' + String(!!v) + '"></span>';
  } else {
    control = '<span class="sq-knob" role="slider" tabindex="0" data-modal-param="' + esc(item.key) + '"
aria-label="' + esc(item.name) + '"'
      + ' aria-valuemin="' + numOr(item.min, 0) + '" aria-valuemax="' + numOr(item.max, 1) + '" aria-val
uenow="' + numOr(v, 0) + '">'
      + '<i style="--a:' + knobAngle(item, v) + 'deg"></i>'
      + '<span class="sq-knob__val">' + esc(fmtValue(item, v)) + '</span></span>';
  }
  return '<div class="sq-param" data-param-cell="' + esc(item.key) + '">' + control
    + '<span class="sq-param__name" title="' + esc(item.key) + '">' + esc(item.name) + '</span></div>';
}
```

```
function bindModalParam(cell, item) {
  if (item.type === 'sample') {
    bindSamplePick(cell, modal.draft[item.key], function (id) {
      setDraft(item.key, id);
```

```
      renderModalParams();
    });
    return;
  }
  if (item.type === 'select' && item.display === 'segment') {
    Array.prototype.forEach.call(cell.querySelectorAll('[data-seg]'), function (b) {
      b.addEventListener('click', function () {
        setDraft(item.key, b.getAttribute('data-seg'));
        renderModalParams();
      });
    });
    return;
  }
  const el = cell.querySelector('[data-modal-param]');
  if (!el) return;
  if (item.type === 'select') {
    el.addEventListener('change', function () { setDraft(item.key, cell.querySelector('select').value);
});
    return;
  }
  if (item.type === 'toggle') {
    // 토글은 끌지 않고 누를 때마다 켜짐 · 꺼짐만 바뀝니다
    const flip = function () {
      const next = el.getAttribute('aria-pressed') !== 'true';
      el.setAttribute('aria-pressed', String(next));
      setDraft(item.key, next);
    };
    el.addEventListener('click', flip);
    el.addEventListener('keydown', function (e) {
      if (e.key === ' ' || e.key === 'Enter') {
        e.preventDefault();
        flip();
      }
    });
    return;
  }
  bindControl(el, {
    item: item,
    value: numOr(v0(modal.draft, item.key), 0),
    render: function (v) { paintKnob(el, item, v); },
    commit: function (v) { setDraft(item.key, v); },
  });
}
```

3.3 값 표시

```
export function fmtValue(item, v) {
  if (!item) return String(v);
  if (item.type === 'toggle') return v ? 'On' : 'Off';
  if (item.type === 'select') return selectLabel(item, v);
  if (item.unit === 'note') return noteName(v);   // Sampler 의 Root 는 노트 이름(C4)으로
  const step = item.step ? item.step : 0;
  const dec = step >= 1 ? 0 : (step >= 0.1 ? 1 : (step >= 0.01 ? 2 : 3));
  const unit = item.unit ? (item.unit === '%' ? ' %' : ' ' + item.unit) : '';
```

```
  return Number(v).toFixed(dec) + unit;
}
```

4. 동작 순서

```
Mode 버튼 클릭 ─▶ 초안(draft.mode)을 바꾸고 renderModalParams 로 칸을 다시 그림
                  ├─ segmentHtml 이 고른 옵션의 hint 를 다시 보여 줌
                  └─ ignoredKeys('oneshot') = loop, sustain, release 칸에 is-ignored (흐리게, 설명 말풍선)

샘플 선택     ─▶ 초안(draft.sample_id)을 바꾸고 칸을 다시 그림 ─▶ Save ─▶ PATCH params 에 sample_id, mode 가 함께 저장
Save 뒤       ─▶ refreshCells 가 카드를 다시 그려 부제가 "샘플 이름, 모드"로 바뀜
```

카드의 모양	조건
부제 `kick_808, Classic`	샘플이 붙어 있음
경고 `⚠ No sample chosen. Notes make no sound.`와 점선 테두리	`sample_id`가 없음
부제 `Sample #12, Classic`	샘플 번호는 있으나 목록에서 이름을 아직 못 찾음 (목록 읽기 전이거나 실패)

카드의 핵심 노브는 `type: "knob"`이고 `primary: true`인 항목만 모으므로, Sample과 Mode는 노브로 나타나지 않고 Gain, Tune, Decay가 올라옵니다.

5. 최신 문법으로 쓰면

```
// segmentHtml 을 최신 문법으로 쓴 모양
const segmentHtml = (item, v) => {
  const cur = (item.options ?? []).find((o) => String(o.value) === String(v));
  const buttons = (item.options ?? []).map((o) => {
    const on = String(o.value) === String(v);
    return `<button type="button" data-seg="${esc(o.value)}" class="${on ? 'is-on' : ''}" aria-pressed="${on}">${esc(o.label)}</button>`;
  }).join('');
  return `<div class="sq-param sq-param--wide sq-segcell" data-param-cell="${esc(item.key)}">…${buttons}…${cur?.hint ? `<p>${esc(cur.hint)}</p>` : ''}</div>`;
};

// One-Shot 에서 무시되는 칸 표시
const ignored = isSampler(modal.plugin) ? ignoredKeys(modal.draft.mode) : [];
ignored.forEach((key) => box.querySelector(`[data-param-cell="${key}"]`)?.classList.add('is-ignored'));
```

전통 문법 (실제 코드)	최신 문법
`filter(...)[0]`	`find(...)`
`'<button ...' + x + '...'`	템플릿 리터럴
`if (cell) { cell.classList.add(...) }`	옵셔널 체이닝 `cell?.classList.add(...)`
`cur && cur.hint ? ... : ''`	`cur?.hint ? ... : ''`

6. 왜 이렇게

선택	이유
새 칸 종류를 스키마의 `type`과 `display`로 표현했습니다	플러그인 이름(`sampler`)을 화면 코드에 박지 않고, 스키마가 모양을 정하게 하는 기존 방식(S1-09 자동 생성)을 지켰습니다. 나중에 Drum Rack이나 다른 악기가 샘플 칸을 쓰면 코드를 고치지 않아도 됩니다.
무시되는 칸 목록(`ignoredKeys`)만 Sampler 전용입니다	"One-Shot에서 Loop, Sustain, Release를 무시한다"는 규칙은 Sampler의 동작에 속하므로 화면의 일반 규칙으로 만들지 않았습니다. 목록이 순수 함수라 시험하기 쉽습니다.
무시되는 칸을 숨기지 않고 흐리게 합니다	칸이 사라지면 모드를 바꿨을 때 노브 배치가 흔들립니다. 흐리게 두고 값은 보존하면 Classic으로 돌아갔을 때 설정이 그대로 있습니다. 목업(`02_oneshot.png`)도 같은 모양입니다.
모드를 바꾸면 칸 전체를 다시 그립니다	힌트 문구와 흐림 표시가 함께 바뀌어야 하고, 초안은 `modal.draft`에 있어서 다시 그려도 값이 사라지지 않습니다. 노브를 만질 때는 다시 그리지 않아 드래그가 끊기지 않습니다.
다른 DB 변경은 없습니다	값은 기존 `track_instruments.params`(JSON)에 `sample_id`와 `mode`로 들어갑니다. 이미 붙어 있던 Sampler는 키가 빠져 있어도 `withDefaults`와 `resolveParams`가 기본값(샘플 없음, `classic`)으로 채웁니다.
카드의 부제는 `values`(기본값을 깐 params)로 만듭니다	저장된 params에 `mode`가 아직 없는 옛 Sampler도 "Classic"으로 표시되어야 합니다.

7. 시험

- `docs/handoff/s1-11c_tests/ui_sampler.mjs` (39건)가 이 문서의 모든 동작을 실제 화면에서 확인합니다.
  - 상세 창은 Sampler로 열리고 탭이 Sample, Amp Env입니다.
  - Mode 줄은 Classic이 눌린 채로 시작하고 Classic 설명이 보입니다. One-Shot을 누르면 설명이 바뀌고, Loop(Sample 탭)와 Sustain, Release(Amp Env 탭)가 흐려지며, Attack은 흐려지지 않습니다. 고치면 "Unsaved changes"가 됩니다.
  - 저장하면 서버의 `params`에 `sample_id`와 `mode`가 들어가고, 카드 부제가 바뀝니다.
  - 샘플을 고른 뒤에도 Mode 선택이 유지됩니다.
- 스크린샷 7장은 `docs/screenshots/2026-10-21_s1-11c_sampler/`에 있습니다.
- 화면 자체의 문제는 시험에서 나오지 않았습니다. 첫 실행의 실패 4건은 시험 기대값의 오류였고(303 참고), 실제 화면은 목업과 같은 모양으로 나왔습니다.